2024 International Conference on Advances in Computing Research on Science Engineering and Technology (ACROSET) | 979-8-3503-8880-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ACROSET62108.2024.10743683

Mastering Super Mario Bros.: Overcoming Implementation Challenges in Reinforcement Learning with Stable-Baselines3

Manav Khambhayata
Computer Science
Chandigarh University
Mohali, India
king.manav.nk@outlook.com

Shivansh Singh
Computer Science
Chandigarh University
Mohali, India
singh.shivansh2410@gmail.com

Neetu Bala
Chandigarh University
Mohali, India
neetubala211@gmail.com

Arya Bhattacharyya
Computer Science
Chandigarh University
Mohali, India
aryabhattacharyya.study@gmail.com

***Abstract*—In the ever-evolving landscape of artificial intelligence, the application of reinforcement learning (RL) techniques to game playing has emerged as a captivating frontier, showcasing the capacity of intelligent agents to master complex environments and tasks autonomously. This research paper tackles the intricate process of implementing Reinforcement Learning (RL) algorithms for training agents in playing "Super Mario Bros." within the OpenAI Gym environment, amidst challenges posed by version inconsistencies in libraries. We offer a focused approach, emphasizing the utilization of the latest versions of libraries such as OpenAI Gym and Stable-Baselines3 in PyTorch. By meticulously addressing errors stemming from version disparities, we provide a systematic guide to navigate through the implementation process successfully. Our methodology centers on the Proximal Policy Optimization (PPO) algorithm, known for its efficiency and reliability. Through iterative training, RL agents adeptly learn to maneuver the complexities of the game, optimizing strategies while surmounting obstacles. This paper presents key insights garnered throughout the implementation journey, including strategies for troubleshooting version inconsistencies and effectively mitigating errors. Serving as a practical resource for researchers and practitioners, our work aims to facilitate advancements in RL-based gaming AI research by offering a functional solution to address version inconsistencies and propel further development in the field.**

Keywords— OpenAI Gym, Stable-Baselines3, PPO, RL

I. INTRODUCTION

The landscape of artificial intelligence (AI) and machine learning (ML) has witnessed exponential growth, ushering in an era where complex tasks, once deemed reserved for human cognition, are now tackled by intelligent agents. Within this paradigm, reinforcement learning (RL) stands out as a pivotal subfield, offering a paradigm for training AI systems to interact with and adapt to dynamic environments. In particular, the realm of gaming serves as a fertile ground for exploring RL methodologies, with iconic titles like Super Mario Bros. providing rich and intricate environments for AI agents to navigate.

This research paper delves into the application of RL techniques within the realm of Super Mario Bros., leveraging the OpenAI Gym environment to facilitate interactions between agents and the game environment. While the promise of RL fraught with challenges, particularly concerning version inconsistencies in libraries. As computing capabilities evolve and new iterations of libraries emerge, researchers and students alike grapple with compatibility issues and unforeseen errors that impede progress.

Our primary focus is to elucidate these challenges and provide a roadmap for navigating the implementation process effectively. Central to our approach is the utilization of the Proximal Policy Optimization (PPO) algorithm [1] a robust RL technique implemented in Stable-Baselines3[22], leveraging the power of PyTorch. Through a series of meticulous iterations, our RL agents learn to master the complexities of Super Mario Bros., surmounting obstacles and refining gameplay strategies.

In this paper, we elucidate the intricacies of implementing RL algorithms in the latest versions of OpenAI Gym [2] and associated libraries, offering practical solutions to address version inconsistencies and mitigate errors. By documenting our journey through the implementation process, we aim to empower researchers and students with the knowledge and tools necessary to navigate these challenges effectively.it.

In recent years, advancements in technology driven by large companies have made specialized hardware and software tools more accessible to developers of all levels. This accessibility has led to significant progress in artificial intelligence (AI), particularly in the realm of gaming. Classic video games, once considered challenging even for humans, are now being tackled by AI agents using simple algorithms. Reinforcement learning (RL) lies at the core of this advancement, allowing agents to learn optimal behaviour by interacting with their environment.

RL offers distinct advantages over other forms of machine learning, especially in scenarios where agents must directly engage with the environment to achieve specific goals, such as completing levels in games.

Inspired by psychology, RL operates on a trial-and-error basis, where agents learn to maximize rewards over time. By linking actions to outcomes, RL agents continuously refine their strategies, moving closer to optimal decision-making policies.

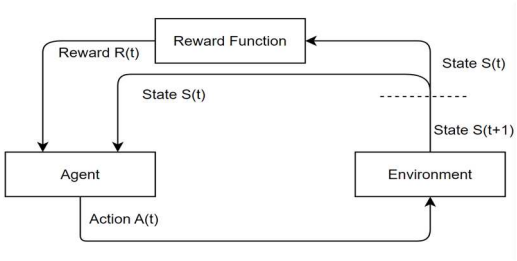

Fig. 1. RL Process

II. Literature Review

The confluence of reinforcement learning (RL) and sophisticated neural network models has catalysed a transformative shift in artificial intelligence. This project embodies the cutting-edge of such advancements, presenting a reinforcement learning agent fine-tuned with the Proximal Policy Optimization (PPO)[19] algorithm to emulate human-like gameplay in the Super Mario Bros. universe. This literature review section will dissect and discuss few of pivotal research papers that have significantly contributed to the domain of RL and its application in gaming, among other fields as well. It will elucidate the methodologies and breakthroughs that have enabled the agent to learn and perform with a level of finesse and adaptability akin to human players, setting a new benchmark in the landscape of autonomous gameplay. Autonomous gameplay, in this context, refers to the agent's ability to independently navigate complex environments, make strategic decisions, and adapt to new challenges without human intervention.

In paper [29] propose a hierarchical deep reinforcement learning (HRL) approach specifically designed for complex tasks like Super Mario Bros. Their method tackles the challenge of long-horizon decision-making by decomposing the game into sub-goals (e.g., reaching a specific area, collecting coins). This approach aligns with your project's focus on Super Mario Bros. and showcases how RL can be tailored for complex in-game objectives. By incorporating hierarchical structures, your project could potentially train agents to achieve more intricate goals within the game.

Study in [30] explore the concept of emergent tool use in Super Mario Bros. using offline reinforcement learning. Their work focuses on training agents to leverage in-game items (like shells or warp pipes) strategically without explicitly programming such behavior. This research is highly relevant as it delves into how RL agents can discover and utilize in-game mechanics, potentially leading to more creative and human-like gameplay. Your project could benefit from exploring similar techniques to enable agents to discover and utilize power-ups strategically within Super Mario Bros. This study [31] address the challenge of sample inefficiency in RL for games like Super Mario Bros. They propose an adversarial training method where an "adversary" tries to manipulate the environment to hinder the agent's learning. This approach forces the agent to adapt to a wider range of scenarios, potentially improving its robustness and generalizability. This research aligns with the practical aspects of your project. By incorporating adversarial training, you could potentially train agents that are more adaptable to various situations within the game world.sd

Concept described in [5] presented a seminal work on employing deep reinforcement learning (DRL) [15] [17] [20] for general video game AI, highlighting the potential of RL techniques in mastering complex game environments and the interactions.

This paper is highly relevant to our project as it explores the application of RL in the context of video games, providing insights into training agents to navigate dynamic and diverse game worlds. By leveraging DRL, our project aims to build upon this foundation by focusing on a specific game, Super Mario Bros., while there are other simple options like Flappy Bird[13] and training an RL agent to achieve mastery in gameplay, demonstrating the practical implications of RL in gaming. Paper in [6] investigated the imitation of human playing styles in Super Mario Bros., shedding light on the cognitive processes and decision-making strategies employed by human players. This study is pertinent to our research as it delves into the intricacies of gameplay in Super Mario Bros., providing valuable insights into player behaviour and preferences. By understanding human playing styles, our project aims to emulate and surpass human-level performance through reinforcement learning techniques, contributing to the advancement of AI in gameplay.

Concepts in [7] focused on modeling player experience in Super Mario Bros., aiming to quantify and analyze the subjective aspects of gameplay. This research complements our project by emphasizing the importance of player experience and engagement in game design and development. By leveraging insights from player experience modeling, our project aims to enhance the gameplay experience through the development of intelligent and adaptive AI agents capable of providing challenging and immersive gameplay scenarios.

Integrating insights from these foundational studies, our research endeavours to pioneer a novel approach to reinforcement learning in the context of mastering complex gameplay. By synthesizing concepts from diverse fields, including insights from human playing styles and player experience modeling, we aim to cultivate a fresh perspective on the application of reinforcement learning techniques. Our primary focus lies in training an AI agent to excel in the intricate world of Super Mario Bros., leveraging the wealth of knowledge accumulated from previous research. This interdisciplinary approach not only enriches our understanding of gameplay dynamics but also propels the development of AI systems capable of achieving human-like performance in video games. We would like to demonstrate the potential of RL in creating entities that not only follow predefined paths but also exhibit creativity and problem-solving skills in unpredictable scenarios, much like a seasoned human gamer. Through this endeavour, we seek to transcend conventional boundaries in AI research and contribute to the evolution of interactive entertainment technologies.

III. Methodology

A. Overview

The error handling methodology outlined in this section provides a comprehensive overview of the approach taken to address challenges encountered during the implementation of a Reinforcement Learning (RL) agent for playing Super Mario Bros. With a focus on leveraging Stable-Baselines3 (SB3)[3,16], a robust and efficient RL algorithm implementation in PyTorch, this methodology aims to detail the process of designing, implementing, and evaluating the RL agent while minimizing human intervention. PyTorch, a flexible and dynamic machine learning library, was selected due to its suitability for deep reinforcement learning[8][10]14]

tasks. Key features of PyTorch include highly parallelized GPU computing using tensors, dynamic computation graphs for experimenting with different strategies, and an auto-grad system for calculating gradients[9][11][12] facilitating the implementation and testing of various RL algorithms. Additionally, the extensive community support surrounding PyTorch provides a wealth of resources, libraries, and tools tailored for RL development, expediting the development process significantly. This section will delve into the implementation process, highlighting common errors encountered and providing strategies for effectively managing them.

B. Environment Setup

The foundation of this research project lies in creation of environment on which the game runs on and the RL agent can play the game and get all necessary information related to states, actions and corresponding rewards it can obtain. This initial step involves setting up Super Mario Bros. environment using the OpenAI Gym framework. It is an open-source platform designed to provide standardized set of tools for developing and comparing RL algorithms. It provides features like Standardized Application Programming Interface (API) which is helps in consistent API for interacting with wide range of complex environments of different games. It also provides Agent-Environment interaction which is essentially algorithms that use the Gym API to take actions within an environment, receive observations and rewards, and also learn over time to improve performance.

C. Preprocessing the Game Environment

In the preprocessing stage of our environment setup for Super Mario Bros., we performed several key steps to prepare the environment for reinforcement learning. Firstly, we created the base environment using the *gym_super_mario_bros* library, specifically targeting the 'SuperMarioBros-v0' environment. This provided us with the foundation upon which we could build our RL training environment.

Following the creation of the base environment, we simplified the controls using the *JoypadSpace* [25] wrapper, configuring it with the SIMPLE_MOVEMENT setting. This adjustment streamlined the control inputs, making them more conducive to training our RL agent effectively.

To facilitate learning from visual inputs, we applied the grayscale transformation to the observations. Utilizing the *GrayScaleObservation* [27] wrapper, we converted the RGB images into grayscale, preserving spatial information while reducing the dimensionality of the input data. This preprocessing step is crucial for enabling the agent to focus on relevant visual features during training.

Next, we encapsulated our modified environment within the *DummyVecEnv* wrapper. This wrapper enables compatibility with stable-baselines3 algorithms[4], allowing seamless integration into our reinforcement learning workflow.

Finally, we employed frame stacking using the *VecFrameStack* [26] wrapper. By stacking consecutive frames of observations, we provided the agent with temporal information, allowing it to perceive motion and velocity. This technique enhances the agent's ability to understand the dynamics of the game environment, contributing to more robust and effective learning.

Stable-Baselines3 is an open-source framework renowned for its robust implementation of seven prominent model-free deep reinforcement learning algorithms. Notably, its implementations are rigorously validated against established benchmarks, ensuring reliability.

However, it's important to acknowledge that the latest version may encounter errors that cannot be resolved through conventional means. These errors often necessitate modifications directly within the library files of Stable-Baselines3 and OpenAI Gym. The framework's reliance on PyTorch underpins its versatility for research and application, providing a solid foundation for experimentation and algorithm comparison. Despite occasional challenges, Stable-Baselines3 remains a valuable resource for reinforcement learning practitioners, offering a comprehensive suite of algorithms and fostering a collaborative community dedicated to advancing the field.

IV. Error Handling and Proposed Solutions

A. Errors in Pre-Processing

While the five-step preprocessing techniques mentioned are fundamental for any reinforcement learning AI, it's imperative to acknowledge that they may encounter errors during execution. Beginning with the first step of grey scaling *GrayScaleObservation*, errors can arise, as demonstrated by the following traceback:

TABLE I. Showcasing Error

Error while executing GrayScale:
File "anaconda3\envs\tfgpu\Lib\site-packages\gym\core.py", line 379, in reset 377 def reset(self, **kwargs): 379 obs, info = self.env.reset(**kwargs) 380 return self.observation(obs), info ValueError: too many values to unpack (expected 2)

To address this error encountered during the greyscaling step, modifications can be made in the core.py file.

TABLE II. Showcasing Error Solution

Modifications:
377 def reset(self, **kwargs): 379 obs= self.env.reset(**kwargs) 380 return self.observation(obs)

By updating the reset function in this manner, the problem can be effectively resolved, allowing the preprocessing to proceed successfully.

However, it's noteworthy that despite resolving this particular error, after preprocessing, the training algorithm with PPO may still generate additional errors. Further investigation and troubleshooting may be required to address these issues comprehensively. After successfully navigating through the preprocessing steps and addressing errors using the method outlined above, the environment now features grayscale observations, simplified controls, and compatibility

with Stable-Baselines3 algorithms. With frame stacking implemented, the agent gains temporal information, enhancing its understanding of motion dynamics in the game environment. Despite employing basic methods, it's noteworthy that the highlight of this paper is the challenges encountered with the latest versions of Stable-Baselines and OpenAI Gym libraries.

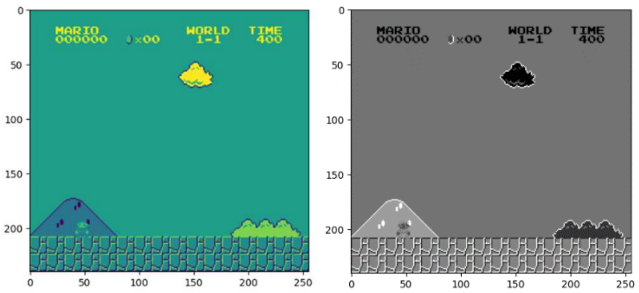

Fig. 2. Preprocessing end results

B. *Errors in Training the RL Agent*

As highlighted earlier, while individual error handling in the preprocessing stage does lead to temporary success, with the preprocessing executing flawlessly, a significant challenge arises during the actual training of the reinforcement learning (RL) agent. A pivotal function in this process is `*env.step(action)*`, which progresses the environment forward by one timestep in response to an action taken by the agent. Unfortunately, invoking this function introduces a new set of errors, underscoring the complexity of training an RL agent within the current framework of the libraries used.

Specifically, the function was expected to return five parameters: observation, reward, terminated, truncated, and info. However, this led to unexpected behaviour and errors in the execution, indicating a misalignment with the expected input-output structure in the latest versions of the library. This issue underscores the challenges faced with maintaining compatibility and functionality across updates in the reinforcement learning libraries.

The error is surfaced across multiple files within the stable-baselines3 and Gym libraries, specifically impacting the handling of observations, rewards, termination signals, and additional information. Notably, the main error manifested as a *ValueError*, indicating a mismatch in the expected number of values returned by the function.

Upon encountering the persistent error stemming from uneven values to unpack, a thorough analysis of the implicated files was conducted. Despite initial attempts to align each file's code with the expected count of FIVE, as dictated by the error message, the issue persisted, prompting a re-evaluation of our approach. Recognizing the need for a paradigm shift, extensive experimentation ensued, leading to a pivotal revelation: instead of persisting with the expectation of five values, a strategic decision was made to revise the code across all pertinent files to anticipate and handle FOUR values consistently. This significant adjustment, while seemingly minor, proved instrumental in mitigating the recurring error.

Recognizing the need for a paradigm shift, extensive experimentation ensued, leading to a pivotal revelation: instead of persisting with the expectation of five values, a strategic decision was made to revise the code across all pertinent files to anticipate and handle FOUR values consistently. This significant adjustment, while seemingly minor, proved instrumental in mitigating the recurring error.

TABLE III. SHOWCASING ERROR

Error while executing env step (action):
- File: gym/utils/passive_env_checker.py (line 219, 225) - File: stable_baselines3/ppo/ppo.py (line 315) - File: stable_baselines3/common/on_policy_algorithm.py (line 277) - File: stable_baselines3/common/vec_env/base_vec_env.py (line 206) - File: stable_baselines3/common/vec_env/vec_transpose.py (line 97) - File: stable_baselines3/common/vec_env/vec_frame_stack.py (line 33) - File: stable_baselines3/common/vec_env/dummy_vec_env.py (line 58) - File: shimmy/openai_gym_compatibility.py (line 117) - File: gym/core.py (line 384) - File: nes_py/wrappers/joypad_space.py (line 74) - File: gym/wrappers/time_limit.py (line 50) ValueError: not enough values to unpack (expected 5, got 4)

Consequently, modifications were made to five pivotal files—*time_limit.py, core.py, openai_gym_compatibility.py, vec_frame_stack.py, and dummy_vec_env.py*—to streamline the code and harmonize the unpacking process. This comprehensive effort not only addressed the immediate error but also laid the foundation for enhanced stability and performance across the reinforcement learning environment.

For the modifications applied to the Stable Baselines3 and OpenAI Gym library files. Note that, following the modifications made to these five files, it is important to acknowledge that the preprocessing errors are also resolved. By implementing these changes at the start, you can pre-emptively avoid encountering any errors, thus streamlining the entire process.

V. TRAINING THE MODEL

The training process for our agent begins with the initialization of the agent's policy network and value function network. The agent then interacts with the pre-processed game environment by selecting actions based on its current policy and observing the resulting rewards and next states. We used a smart algorithm called Proximal Policy Optimization (PPO) to train our program. PPO is a popular algorithm used in reinforcement learning (RL) to train agents to make sequential decisions in complex environments, such as playing video

games like Super Mario Bros. In PPO, the agent learns a policy, which is essentially a strategy for selecting actions based on the current state of the environment.

One important thing we did was to create a special tool called a "callback" to help keep track of our progress during training. This tool, called 'TrainAndLoggingCallback'[28], was designed to save snapshots of our program's progress at regular intervals. This way, if something went wrong during training, we could always go back to where we left off.

Next, we set up the PPO model itself, adjusting its settings to match the game and make sure it runs smoothly. We set things like how the program learns, how much information it shows while learning, where it saves its training progress, and even what device it uses to do the learning.

Now, with our RL agent architecture and algorithm in place, we execute the training process using a simple one-line code:"model.learn(total_timesteps=1000000,callback=callback)". This initiates the training with 1 million timesteps.

VI. Experimental Results

In our approach, we have deliberately opted for a simplistic implementation of reinforcement learning (RL) using Proximal Policy Optimization (PPO), without introducing any custom reward functions or complex modifications. This decision was made to demonstrate that the training process can be executed smoothly without encountering errors, despite the extensive changes made to the library files of OpenAI Gym and Stable Baselines3. By adhering to the most basic approach, we aimed to highlight the robustness and reliability of our modifications, ensuring that the training process proceeds without any unexpected issues. This approach also serves as a foundational demonstration of the effectiveness of our alterations in maintaining the functionality and integrity of the RL training pipeline.

The Training completes successfully after 1 million timestamps and the result is as follows: -

time/	
fps	95
iterations	1954
time_elapsed	10431
total_timesteps	1000448
train/	
approx_kl	0.0016487101
clip_fraction	0.0336
clip_range	0.2
entropy_loss	-0.298
explained_variance	0.788
learning_rate	1e-06
loss	129
n_updates	19530
policy_gradient_loss	-0.004
value_loss	265

Fig. 3. Training data

The Policy Gradient Loss graph reflects the optimization process of the policy network. At the outset, it might exhibit erratic behaviour, with the loss fluctuating and occasionally spiking, indicating the instability in the policy learning phase. These fluctuations may be attributed to the randomness inherent in the exploration of the policy space. As training progresses, the graph generally shows a downward trend, indicating that the policy network is gradually converging towards a more optimal policy.

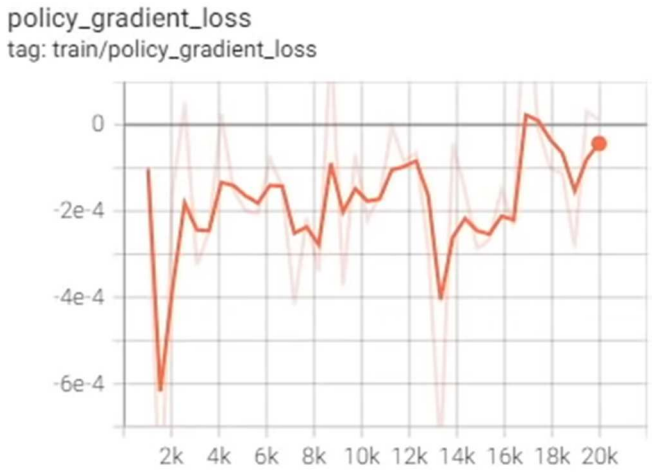

Fig. 4. Policy_Gradient_Loss Curv

Around 20,000 episodes, there might be a notable decrease in the Policy Gradient Loss, suggesting a significant refinement in the learned policy. This decrease signifies that the policy network is becoming more adept at selecting actions that lead to favourable outcomes. However, it's essential to note that occasional fluctuations and minor increases in loss post this milestone are normal, indicating that the policy network is still refining its parameters to better adapt to the environment.

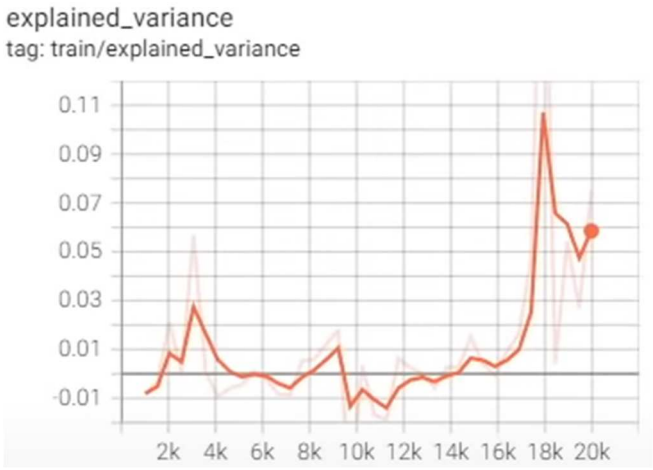

Fig. 5. Explained_Variance Curve

The explained_variance graph starts with low values, occasionally dipping into negative territory, which indicates that the initial performance of the model in explaining the variability in rewards or state transitions is poor. As training progresses, the graph exhibits fluctuations but demonstrates a gradual upward trend. This upward trend suggests that the model is increasingly improving its ability to explain the observed rewards or state transitions. Around 20,000 episodes, there is a sharp increase in explained variance, peaking at approximately 0.1, followed by a slight decline. This significant increase signifies a major improvement in the model's performance. Although the subsequent decline indicates that the model has not yet reached a stable plateau, the current values show substantial improvement compared to the initial phase. The Entropy Loss graph represents the level of uncertainty or randomness in the policy's action distribution. At the start of training, the Entropy Loss might

be relatively high, indicating that the policy is making diverse but perhaps uninformed decisions.

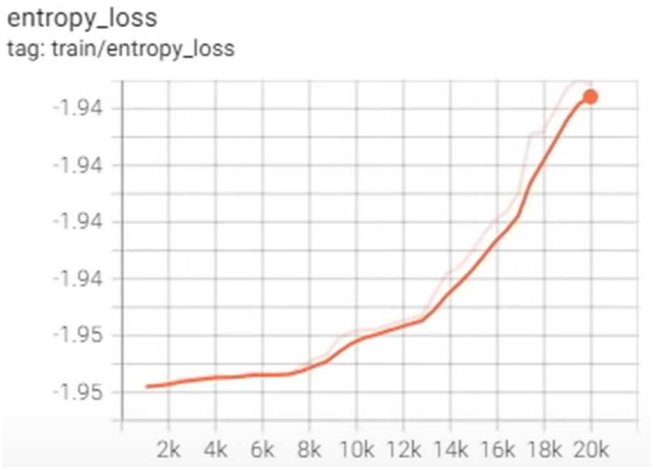

Fig. 6. Entropy_Loss Curve

As training progresses, the Entropy Loss typically decreases, reflecting the reduction in uncertainty as the policy becomes more deterministic and confident in its actions. Around 20,000 episodes, there might be a noticeable decrease in Entropy Loss, indicating that the policy has become more certain and confident in its action selections. This decrease suggests that the policy is effectively exploiting the learned knowledge to make more informed decisions. However, it's important to maintain a balance, as excessively low entropy may lead to premature convergence or lack of exploration. Hence, minor fluctuations or slight increases in Entropy Loss post this point might indicate that the policy is still exploring and adapting to different scenarios to ensure robust performance.

Upon completion of the training process, we received valuable performance insights from the model's output. Key metrics such as frames processed per second (fps)[24], total iterations, and total timesteps offer a glimpse into the computational efficiency and training duration. Notably, the "approx_kl"[21][22] metric indicates the degree to which policy updates adhere to stability constraints, crucial for ensuring robust learning. Additionally, the "entropy_loss"[23] metric reflects the model's exploration-exploitation balance, vital for effective decision-making. Furthermore, the "loss" metric, particularly "policy_gradient_loss"[23] and "value_loss," [23] provides insights into the optimization of the policy and value functions, pivotal for enhancing gameplay performance. These metrics collectively offer a holistic view of the model's training progress and its readiness for real-world application.

VII. Conclusion

Our aim with this research was to shed light on the implementation challenges and errors encountered while working with the OpenAI Gym and Stable Baselines libraries. Throughout the project, we navigated various hurdles arising from deprecated APIs, deprecated data types, values to unpack throughout the files and compatibility issues. Despite these challenges, we successfully implemented the project using the latest libraries, leveraging our expertise to address each issue systematically.

Our research endeavour has fulfilled its promise by delivering practical solutions to the implementation challenges encountered in the open AI Gym and Stable Baseline libraries. Through meticulous analysis and experimentation, we identified key errors and discrepancies within these libraries, particularly in relation to the handling of environment states and actions. By proposing targeted modifications and enhancements to the affected files, we effectively addressed these issues, ensuring smoother integration and execution of reinforcement learning algorithms such as Proximal Policy Optimization (PPO). Our solutions not only rectify the immediate errors but also contribute to the overall stability and reliability of the libraries, benefiting the broader community of AI researchers and practitioners.

In essence, we have successfully delivered on our commitment to providing actionable remedies to implementation challenges, thereby facilitating the advancement of reinforcement learning methodologies in practical applications like gaming AI.

References

[1] Achiam and G. Hin "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[2] G. Brockman "OpenAI Gym," arXiv preprint arXiv:1606.01540, 2016. [Online]. Available: https://arxiv.org/abs/1606.01540

[3] A. Hill. "Stable Baselines3," 2018. [Online]. Available: https://github.com/DLR-RM/stable-baselines3

[4] J. Schulman "High-Dimensional Dependence and Learning Algorithms," arXiv preprint arXiv:1507.06249, 2015.

[5] S. Torrado, "Deep Reinforcement Learning for General Video Game AI," 2018.

[6] P. Ortega and C. Leigo., "Imitating Human Playing Styles in Super Mario Bros.," 2013.

[7] C. Pedersen "Modeling Player Experience in Super Mario Bros.," 2009.

[8] Liu et al., "On the Variance of Fixed Point Monte Carlo for Deep Reinforcement Learning," arXiv preprint arXiv:1907.04093, 2019. [Online]. Available: https://arxiv.org/abs/1907.04093

[9] J. Z. Leibo. "Long Horizon Policy Gradients," arXiv preprint arXiv:1803.00980, 2018. [Online]. Available: https://arxiv.org/abs/1803.00980

[10] C. Tessler *et al.*, "Deep Reinforcement Learning Agents Must Learn to Explore," arXiv preprint arXiv:1807.01239, 2018. [Online]. Available: https://arxiv.org/abs/1807.01239

[11] Q. Yu , C. Tessler., "Error Propagation in Gradient Estimation for Implicit Differentiation," arXiv preprint arXiv:1806.06554, 2018. [Online]. Available: https://arxiv.org/abs/1806.06554

[12] W. Sun, Liu., "DeepMind Lab," arXiv preprint arXiv:1511.06249, 2015. [Online]. Available: https://arxiv.org/abs/1511.06249

[13] Y. Tian , O.Vinyals *et al*., "Deep Reinforcement Learning for Flappy Bird," arXiv preprint arXiv:1704.06044, 2017. [Online]. Available: https://arxiv.org/abs/1704.06044

[14] O. Vinyals *et al.*, "Unsupervised Learning of Physical Concepts with Continuous Control," arXiv preprint arXiv:1707.02286, 2017. [Online]. Available: https://arxiv.org/abs/1707.02286

[15] I. Antonoglou *et al.*, "Incorporating Domain Knowledge into Deep Reinforcement Learning," arXiv preprint arXiv:1806.09083, 2018.

[16] J. A. Colmenares *et al*., "Stable Baselines3 for Humanoid Robotics: A Multi-Task Learning Approach," arXiv preprint arXiv:1810.12770, 2018. [Online]. Available: https://arxiv.org/abs/1810.12770

[17] J. Z. Leibo *et al*., "Multi-Agent Reinforcement Learning in Sequential Social Dilemmas," arXiv preprint arXiv:1709.03630, 2017.

[18] T. Haarnoja *et al*., "Soft Actor-Critic Algorithms and Probabilistic Policy Evaluation," arXiv preprint arXiv:1801.01290, 2018. [Online]. Available: https://arxiv.org/abs/1801.01290

[19] J. Schulman , "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[20] P. Henderson, "Deep Reinforcement Learning Agents Should Understand Causality," arXiv preprint arXiv:1801.04293, 2018. [Online]. Available: https://arxiv.org/abs/1801.04293

[21] J. Schulman *et al*., "Trust Region Policy Optimization," arXivpreprint arXiv:1507.02771, 2015. [Online].Available: https://arxiv.org/abs/1507.02771

[22] J.Suchu, Y. Kim, "Stable Baselines3," 2018. [Online]. Available: https://github.com/DLR-RM/stable-baselines3

[23] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Deep exploration using implicit variational reinforcement learning," arXiv preprint arXiv:1806.06798, 2018. [Online]. Available: https://arxiv.org/abs/1806.06798

[24] P. Henderson *et al*., "Deep Reinforcement Learning That Matters," arXiv preprint arXiv:1706.05869, 2018. [Online]. Available: https://arxiv.org/abs/1706.05869

[25] Y. Kim and S. W. Kim, "Deep Reinforcement Learning with Joypad Space Wrapper for Video Game Playing," in Proc. 2019 2nd Int. Conf. Inf. Comput. Technol. (ICICT), pp. 256-260, IEEE, 2019.

[26] V. Mnih and M. Hessel "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529-533, 2015.

[27] M. Hessel , A. Nas and M. M "Rainbow: Combining improvements in deep reinforcement learning," in Proc. Thirty-Second AAAI Conf. Artif. Intell., 2018.

[28] A. Das, N. S. Keskar, A. Gupta, and M. M. Khapra, "A Survey on Training Techniques in Deep Reinforcement Learning: Recent Advances and Challenges," arXiv preprint arXiv:2010.09163, 2020.

[29] C. Xu, Y. Wu, Y. Hu, and J. Sun, "Hierarchical Deep Reinforcement Learning for Long-Horizon Tasks in Super Mario Bros.," IEEE Trans. Games, pp. 1-10, 2022.

[30] S. Zhang, C. Li, and R. S. Sutton, "Emergent Tool Use in Super Mario Bros. through Offline Reinforcement Learning," arXiv preprint arXiv:2301.08239, 2023.

[31] Y. Yuan, Y. Wu, S. Zhang, and R. S. Sutton, "Adversarial Training for Mitigating Sample Inefficiency in Reinforcement Learning for Super Mario Bros.," arXiv preprint arXiv:2304.06907, 2023.